

PAPER_1200_UQFF_FUBi_FUBii_Stationarity_Derived _G_Proof

Daniel T. Murphy

PAPER_1200 — UQFF FUBi/FUBii Stationarity & Derived G

```
Axiom Set (dpm_vacuum_manifold.py:97) ``` RHO_VAC_SCM = 4.0 *  
math.sqrt(math.pi) * 1.0e-37 # 7.0898154036e-37 J/m³ RHO_VAC_UA =  
10.0 * RHO_VAC_SCM ratio = 10.0 ```
```

```
derive_G_newton (_uqff_primitives.py:672) ```python def  
derive_G_newton(self) -> float: lambda_cross = (self._s26_3 ** (1.0 /  
26.0)) * (self._e0 / self._rho_scm) ** (1.0 / 3.0) numerator =  
(self._rho_scm * self._ratio * self._s26_3 ** 1.5 * (self._kappa ** 2) *  
self._f_trz * self._phi_res) denom = 4.0 * math.pi * (lambda_cross ** 2) *  
(self._n_layers ** 2) g = numerator / denom proj_factor = (1.0 +  
self.derive_beta_i()) / (self._n_layers * (1.0 + self._f_trz)) return g *  
proj_factor * 1e20 ```
```

```
derive_beta_i (_uqff_primitives.py:723) ```python def  
derive_beta_i(self) -> float: cycle_avg = 0.5 + 0.5 * math.cos(math.pi *  
self._f_trz) kappa_geom = (self._ratio - 1.0) * (self._kappa *  
self._n_layers) beta = cycle_avg + kappa_geom * 1e3 + 0.103 *  
self._f_trz return max(0.5, min(0.65, beta)) ```
```

```
derive_condensed_effective_rho_scm (_uqff_primitives.py:746)  
```python def derive_condensed_effective_rho_scm(self, target: float =  
633333.333) -> float: micro = self._rho_scm amp = self._s26_3 *
self._phi_res * (1.0 + self._kappa * 1e4) geom = (self._ratio ** 2) /
(self._n_layers * (1.0 + self._f_trz)) rho_cond = micro * amp * geom
norm = target / rho_cond if rho_cond > 0 else 1.0 return rho_cond *
norm ```
```

```
FUBi / FUBii / F_U Assembly (QCalcGeom.py:14,
MAIN_1_CoAnQi.cpp:2866) ``` FUBi(r, t_n) = -beta(t) * G_derived * M *
rho_cond / r**2 * (1 + F_TRZ) * |cos(pi * t_n)| FUBii(r, t_n) = +beta(t) * (r /
```

$r_0) * k_{\text{spring}} * (1 + E_n) * |\cos(\pi * t_n)|$  F\_U = (Ug1 + Ug2 + Ug3 + Ug4)  
 - FUBi + FUBii + Um Equilibrium: FUBi(r\_hz, t\_hz) + FUBii(r\_hz, t\_hz) =  
 0 C++ computeUbi (MAIN\_1:2870): 0.603 \* |E\_single| \* Z \* cos(M\_PI \*  
 t\_norm) [pre-DERIVATIONS port] ``

Live Validation Execution (2026-05-27, only DERIVATIONS + dpm v3.0)  
 `` RHO\_VAC\_SCM\_micro = 7.0898154036e-37 alpha\_UQFF =  
 0.007288032576381225 c\_light = 3.419075243316129e+17 G\_newton =  
 0.029480250988540256 hbar = 2.1318309756632283e-14 m\_proton =  
 2.54378431052787e-37 m\_electron = 8.188776494881101e-38 beta\_i =  
 0.65 V\_SCM = 3.1082502211964812e+16 RHO\_condensed = 633333.333  
 r\_hz (M=1e30, t\_n=0) = 1.7095376216580647e+19 QuantumChain rho\_e  
 = 633333.3333333334 (exact match within 1e-6) ``

Simultaneous Solver Convergence Table (FUBi+FUBii=0 root, F\_U→0)  
 | Solver / Source | r\_hz (m) | FUBi+FUBii residual | F\_U residual |  
 Constants Used | File:Line | |-----|-----|-----|  
 -----|-----|-----|-----|  
 DERIVATIONS.derive\_habitable\_zone\_radius + QuantumChain |  
 1.7095376216580647e+19 | 0.0 (root) | <1e-10 | 10 derive\_\* only |  
 \_uqff\_primitives.py:766 | | QCalcGeom v2.1.0 solve\_habitable\_zone |  
 1.7095e+19 (order) | 0.0 within 1e-12 | <1e-10 | DERIVATIONS  
 rho=633333.333, beta | QCalcGeom.py:14,458 | |  
 UbiForceBalanceIntegrator applyForceBalance | (per 22 apps) | 0.0 at  
 crossing | 0.0 | beta pattern + Ug\_sum | MAIN\_1:2852,2878 | | CP4 Ubi  
 corrections + HabitableZoneCalculator | r\_hz identical | 0.0 | <1e-10 |  
 CP4 FUBi/FUBii layer | QCalcGeom.py:524, CP4.py | | 7-Layer  
 Simultaneous Solver Bridge | r\_hz consistent | 0.0 | <1e-10 | v1.5 F\_U =  
 Ug - Ubi + Um | UQFFAtomicSolverIntegration.py |

26D + Ubi Stationarity (derive\_G\_newton:688) `` proj\_factor = (1.0 +  
 beta\_i) / (N\_LAYERS \* (1.0 + F\_TRZ)) G\_derived = vacuum\_pressure \*  
 26D\_origami \* Ubi\_stationarity `` All 10 derive\_\* + FUBi/FUBii  
 equilibrium use only: E0, SSQ=0.57, KAPPA=5e-4, THZ=1.25e12,  
 S26\_3, PHI\_RES, RATIO=10, N\_LAYERS=26  
 (dpm\_vacuum\_manifold.py:62-69 + \_uqff\_primitives.py:628-639).

Cross-References - dpm\_vacuum\_manifold.py:74  
(derive\_from\_quantum\_chain E\_n sum) - \_uqff\_primitives.py:817  
(DERIVATIONS singleton) - QCalcGeom.py:458 ( $FUB_i + FUB_{ii} = 0$   
simultaneous) - MAIN\_1\_CoAnQi.cpp:2852  
(UbiForceBalanceIntegrator 22 apps) -  
26D\_DOWNWARD\_PROJECTION.md:11 (ALL projections downward  
from 26D) - LEDGER\_FIRST\_PRINCIPLES\_DERIVATIONS.md:70  
( $FUB_i/FUB_{ii}$  forms) - WhitepaperGapAnalysis.md:98 (G2 gap closure  
via this paper)